Plan de pruebas — Evento Cultural

Estrategia, alcance y criterios de verificación (artefacto 7 del marco ágil)

Roberto Riffo · José Miguel Moncada (QA) · David Herrera

29 de septiembre de 2026

Proyecto Capstone APT122 · Duoc UC · Sede Maipú

1. Propósito

Este documento define la estrategia de pruebas del proyecto **Evento Cultural**. Su finalidad es establecer qué componentes deben verificarse, con qué enfoque, en qué entornos y bajo qué criterios generales. El historial operativo de ejecuciones, resultados e incidencias se conserva en la matriz `Matriz_Casos_de_Prueba_Evento_Cultural.xlsx` y en las evidencias de cada ciclo de prueba. Este documento incorpora únicamente resúmenes de ciclos cerrados para mantener la trazabilidad metodológica sin duplicar el registro detallado.

El plan cubre la plataforma web y móvil, el pipeline de extracción de eventos, la persistencia en Supabase, la automatización mediante GitHub Actions y las integraciones necesarias para entregar un catálogo cultural vigente, normalizado y trazable.

2. Objetivos de calidad

- comprobar que las funciones priorizadas en la matriz MoSCoW cumplen sus criterios de aceptación;
- verificar que el catálogo no publique eventos vencidos, duplicados, irrelevantes o sin trazabilidad mínima;
- asegurar que la integración scraper–Supabase–frontend conserve la integridad de los datos;
- validar seguridad, rendimiento, accesibilidad, compatibilidad y capacidad de recuperación;
- detectar regresiones antes de integrar cambios en la rama principal o desplegar;
- producir evidencia reproducible, vinculada con requisitos e historias de usuario.

3. Alcance, supuestos y exclusiones

3.1 Alcance incluido

Se incluyen los RF-01 a RF-16, los RNF-01 a RNF-12 y las historias HU-01 a HU-15, según su prioridad y disponibilidad funcional. RF-17 se conserva como prueba futura por estar fuera de la primera versión. Las funciones todavía no desarrolladas pueden tener casos diseñados, pero no deben declararse aprobadas hasta que exista una implementación verificable.

3.2 Supuestos de planificación

- La integración del frontend con Supabase se considera disponible para diseñar y ejecutar pruebas de catálogo, autenticación, interacciones y RLS.
- El catálogo dispone de eventos vigentes, vencidos, gratuitos, pagados, con y sin coordenadas, pertenecientes a distintas regiones y categorías.
- Las credenciales privadas se entregan mediante variables de entorno o secretos de GitHub y nunca se incorporan a documentos, capturas ni archivos de prueba.
- La integración cartográfica requiere la configuración posterior del proveedor y de su clave de API. Sus casos permanecen planificados y se ejecutarán cuando esa dependencia esté disponible.

3.3 Exclusiones actuales

No se incluyen pruebas contractuales de cobro, publicidad, pagos, analítica comercial ni estadísticas para organizadores. Tampoco se validan rutas de transporte público mientras no exista una integración específica. Las pruebas de publicación por organizadores se limitan al diseño hasta que se defina la relación de propiedad entre cuentas y organizadores.

4. Componentes y subplanes de prueba

Código	Componente	Propósito del subplan
PP-01	Conectores del scraper	Verificar extracción desde HTML, REST, calendarios y navegación dinámica; paginación, fuentes estacionales y aislamiento de fallos.
PP-02	Normalización y calidad	Validar fechas, texto, ubicación, comuna, región, organizador, categorías, imágenes, URL oficial y procedencia.
PP-03	Duplicados y ciclo de vida	Comprobar claves deterministas, idempotencia, actualización, vencimiento, desaparición, bloqueo y reconciliación.
PP-04	OCR e inteligencia artificial	Validar afiches, respuestas vacías, timeout, cuota, reintentos, cola pendiente, parafraseo y procesamiento selectivo.
PP-05	Checkpoints y recuperación	Interrumpir y reanudar fuentes y lotes sin perder progreso ni repetir trabajo consolidado.
PP-06	Supabase y modelo de datos	Verificar tablas, relaciones, restricciones, staging, RPC, upsert, RLS, Auth y separación entre datos públicos, privados y administrativos.
PP-07	GitHub Actions	Verificar ejecución manual y programada, secretos, caché, artefactos, límites de tiempo, reanudación y notificación de fallos.
PP-08	Catálogo y navegación	Probar listado, búsqueda, filtros, orden, estados vacíos, paginación, detalle y redirección a la fuente oficial.
PP-09	Interfaz responsive y móvil	Verificar navegación, contenido y controles entre 320 px y escritorio, además del empaquetado móvil aplicable.
PP-10	Mapas y geolocalización	Validar marcadores, coordenadas, eventos sin georreferencia, apertura de ficha y manejo de clave/configuración del proveedor.
PP-11	Usuarios e interacciones	Probar registro, inicio y cierre de sesión, perfiles, señales de interés/desinterés, reportes y aislamiento por usuario.
PP-12	Recomendaciones	Validar elegibilidad, pesos, afinidad por categoría y zona, señales negativas, vigencia y orden estable.
PP-13	Seguridad	Evaluar RLS, roles, propiedad, exposición de secretos, acceso anónimo, validación de entradas y enlaces externos.
PP-14	Rendimiento y escalabilidad	Medir consultas, filtros, carga inicial, pipeline por lotes, uso de memoria y comportamiento con catálogos crecientes.
PP-15	Accesibilidad y usabilidad	Revisar WCAG 2.2 AA, teclado, foco, lector de pantalla, contraste, comprensión y tareas con usuarios objetivo.

Código	Componente	Propósito del subplan
PP-16	Despliegue y portabilidad	Verificar configuración reproducible en GitHub Actions, Vercel y Capacitor. Docker no se utiliza por decisión del equipo: el despliegue sin contenedores resulta más simple de implementar y mantener trabajando con las capas gratuitas de las herramientas del proyecto; su plan queda documentado como alternativa futura, aplicable solo si la estrategia cambiara hacia un servidor propio con modelos de IA especializados.

5. Niveles y tipos de prueba

Nivel o tipo	Aplicación en el proyecto
Unitaria	Funciones puras, parsers, normalizadores, filtros, servicios, modelos y reglas de recomendación.
Componente	Cada conector, servicio Angular, repositorio, exportador y política de acceso se prueba de forma aislada.
Integración	Comunicación entre scraper, NVIDIA NIM, Supabase, Auth, frontend, Maps y GitHub Actions.
Extremo a extremo	Recorrido desde una fuente pública hasta la visualización y apertura de la fuente oficial por el usuario.
Regresión	Repetición de la suite automatizada y de los recorridos críticos tras cambios de código, esquema o fuente.
Aceptación	Validación de criterios asociados a RF, RNF e historias de usuario con representantes del equipo y usuarios objetivo.
Rendimiento	Latencia, percentil 95, volumen, memoria, duración del pipeline y comportamiento bajo crecimiento.
Seguridad	Autorización, RLS, secretos, sesiones, roles, propiedad, entradas inválidas y exposición de datos.
Accesibilidad	Evaluación automática y manual de WCAG 2.2 AA, teclado, foco, semántica y lector de pantalla.
Compatibilidad	Navegadores, resoluciones, orientación, web móvil y empaquetado Android cuando corresponda.
Resiliencia	Errores 403/404/429/5xx, timeout, fuente vacía, cambios HTML, caída de servicios y recuperación desde checkpoints.

6. Estrategia por área

6.1 Scraper y fuentes

Cada familia de conectores debe probarse con contenido válido, vacío, incompleto y malformado. Un fallo de una fuente no debe interrumpir las demás. Se verifican paginación completa, control de velocidad, detección de cambios, fuentes estacionales, URLs `webcal`, respuestas 403/404 y reutilización incremental.

6.2 Calidad, consolidación y vigencia

Las pruebas deben cubrir fechas únicas y rangos, eventos permanentes, zonas horarias, texto con shortcodes/CSS, direcciones aproximadas, múltiples categorías, organizadores bloqueados, URLs caídas y eventos eliminados por su fuente. La reejecución sobre la misma entrada no debe duplicar registros.

6.3 OCR, enriquecimiento y cuotas

Se emplean dobles de prueba para la mayoría de los casos y muestras controladas para la API real. Se comprueban timeouts, `Retry-After`, errores transitorios, límite de 40 RPM, agotamiento de reintentos, persistencia de la cola y omisión de IA para eventos sin cambios. Ninguna prueba masiva debe consumir la cuota sin autorización del equipo.

6.4 Supabase, autenticación y seguridad

Las pruebas utilizan usuarios anónimo, autenticado, editor y administrador. Cada rol intenta operaciones permitidas y prohibidas. Se verifican relaciones, restricciones, RLS, RPC, publicación desde staging, reconciliación, bloqueo de organizadores y ausencia de claves privadas en el cliente o en los artefactos.

6.5 Frontend, móvil y mapas

Los recorridos críticos son descubrir, buscar, filtrar, abrir detalle, reconocer fecha y ubicación, consultar organizador y abrir la fuente oficial. Se prueban estados de carga, error, vacío y datos parciales. Los casos de Maps se diseñan desde ahora, pero su ejecución requiere un proveedor cartográfico configurado y coordenadas de prueba. La implementación actual utiliza Leaflet con mosaicos de OpenStreetMap, por lo que no depende de una clave privada del cliente.

6.6 Recomendaciones e interacciones

Se generan interacciones controladas para confirmar que señales positivas y negativas modifican la prioridad sin exponer datos entre usuarios. Los eventos vencidos, bloqueados o no publicados nunca son elegibles. Debe existir un resultado de respaldo cuando un usuario nuevo no tenga historial.

6.7 Rendimiento, accesibilidad y usabilidad

Con un catálogo de referencia de al menos 1.000 eventos, las consultas paginadas de listado y filtros deben registrar p95 menor o igual a 2 segundos en el entorno acordado. La accesibilidad se evalúa con herramientas automáticas y revisión manual. La usabilidad se valida mediante tareas concretas con usuarios representativos y registro de hallazgos.

7. Entornos y datos de prueba

Elemento	Definición
Entorno local	Ejecución de pruebas unitarias, componentes, frontend y validaciones rápidas con variables de entorno de prueba.
Integración	Proyecto Supabase no productivo o datos aislados mediante identificadores de prueba; usuarios y políticas equivalentes al diseño objetivo.

Elemento	Definición
CI	GitHub Actions con secretos de prueba, caché, artefactos y límites equivalentes al workflow real.
Frontend desplegado	Entorno de revisión en Vercel o equivalente, sin credenciales administrativas en el navegador.
Datos mínimos	Eventos vigentes/vencidos, gratuitos/pagados, con/sin imagen, con/sin coordenadas, múltiples categorías, distintas regiones, duplicados y organizadores bloqueados.
Datos inválidos	Fechas imposibles, HTML contaminado, URL caída, respuesta vacía, coordenadas fuera de Chile, texto excesivo y referencias inexistentes.
Privacidad	Se utilizan cuentas y datos sintéticos; no se incorporan datos personales reales en capturas, logs ni planillas.

8. Criterios de control

8.1 Entrada

Un ciclo puede comenzar cuando existe una versión identificable, el entorno responde, las dependencias necesarias están disponibles, los datos de prueba fueron preparados y los criterios de aceptación son verificables. Los casos dependientes de Maps requieren además una clave restringida y una configuración aprobada.

8.2 Suspensión y reanudación

La ejecución se suspende si el entorno es inestable, existen migraciones incompatibles, faltan secretos, se alcanza una cuota externa o los datos de prueba quedaron corruptos. Se reanuda después de documentar la causa, restaurar el entorno y comprobar con una prueba de humo que el flujo básico vuelve a operar.

8.3 Salida

El ciclo termina cuando los casos críticos y altos fueron ejecutados, no quedan defectos críticos abiertos, las regresiones están controladas y cada resultado posee evidencia. Las excepciones aceptadas deben registrar responsable, justificación y riesgo residual.

9. Gestión de casos, defectos y evidencia

La matriz Excel es la fuente operativa de los casos. Cada caso posee identificador único, requisito, precondiciones, datos, pasos, resultado esperado, prioridad y dependencia. Las ejecuciones se registran sin sobrescribir el historial. Un defecto debe indicar severidad, prioridad, ambiente, caso que lo detectó, pasos de reproducción, evidencia y criterio de cierre.

La evidencia puede consistir en salida de pruebas automatizadas, URL de GitHub Actions, captura, log sanitizado, consulta SQL, reporte de accesibilidad o acta de prueba con usuarios. Las claves, tokens y datos personales deben ocultarse antes de adjuntar evidencia.

10. Ejecución parcial de Fase 2

10.1 Frontend desplegado

El 11 de octubre de 2026 se ejecutó un ciclo exploratorio y funcional sobre el despliegue de producción en https://eventocultural.vercel.app. La muestra cubrió el flujo de descubrimiento en escritorio (1440×900) y móvil (390×844): inicio, búsqueda, estado vacío, filtro regional, detalle, mapa, calendario, navegación inferior, recomendaciones de invitado y perfil sin sesión. La versión evaluada se vinculó con la rama `deploy/vercel-preview` y el commit `4da3963`; la asociación de esta rama con producción fue informada por el equipo y el historial público del repositorio quedó capturado como evidencia.

Indicador	**Resultado registrado**
Escenarios ejecutados	13; corresponden a 12 casos canónicos de la matriz porque la búsqueda por título se verificó en escritorio y móvil.
Resultado	6 aprobados, 6 aprobados con observaciones y 1 fallido. En la matriz, los aprobados con observaciones se registran como *Aprobado* y conservan el defecto asociado en observaciones.
Defectos	13 hallazgos abiertos: 2 de severidad alta, 8 media y 3 baja.
Bloqueadores principales	El mapa móvil no renderiza todos sus mosaicos y la barra inferior fija cubre contenido en distintas vistas.
Rendimiento observado	Cinco solicitudes del documento HTML de inicio respondieron HTTP 200; promedio 0,409 s, mínimo 0,239 s y máximo 0,575 s. Esta medición no sustituye Core Web Vitals.
Evidencia	Matriz `Matriz_Casos_de_Prueba_Evento_Cultural.xlsx` y carpeta `Evidencias_QA_Frontend_2026-10-11/`.

10.2 Automatización del scraper

También se revisaron ejecuciones reales del repositorio del scraper en GitHub Actions. La matriz conserva cinco ejecuciones asociadas a casos canónicos de integración continua.

Indicador	**Resultado registrado**
Run programado principal	Run #25, rama `main`, commit `bd7a052`: estado *Success*, duración 13 min 58 s y dos artefactos, correspondientes a los estados `extract` y `final`.
Secuencia del pipeline	Validación de secretos, pruebas, restauración de estado, extracción, enriquecimiento y sincronización terminaron correctamente. Las pruebas se ejecutaron antes de publicar.
Validación de fuentes	Workflow manual *Validar inventario de fuentes culturales*, run #1, commit `0498e63`: estado *Success*, duración 1 min 14 s y un artefacto de auditoría.

Indicador	Resultado registrado
Continuidad observada	Los runs programados #18 a #25 aparecen exitosos; el run #12 conserva el antecedente de una falla de pruebas que bloqueó las fases posteriores.
Puntualidad del cron	El run #25 comenzó a las 06:16 de Chile, fuera de la ventana nocturna esperada para el cron de las 23:37. El pipeline se considera funcional, pero `CP-CI-002` queda fallido y se registra `DEF-CI-001`.
Evidencia	Carpeta `Evidencias_CI_Scraper_2026-10-11/` y matriz operativa.

El flujo principal es demostrable, pero el mapa móvil no se considera aprobado mientras permanezcan abiertos los dos defectos de severidad alta. Autenticación, persistencia de preferencias, seguridad/RLS, accesibilidad exhaustiva, compatibilidad entre navegadores, Core Web Vitals y regresión posterior a las correcciones permanecen planificados para la Fase 3; no se declaran ejecutados en este ciclo.

11. Roles y responsabilidades

Rol	Responsabilidad
QA / base de datos	Mantener el plan y la matriz, preparar datos, coordinar ejecuciones, validar Supabase/RLS y consolidar evidencias.
Frontend / UX	Diseñar y ejecutar casos funcionales, responsive, accesibilidad y usabilidad; corregir defectos de interfaz.
Backend e integraciones	Validar contratos, autenticación, Maps y servicios; apoyar pruebas de integración y seguridad.
Responsable del scraper	Mantener pruebas de conectores, normalización, IA, checkpoints, deduplicación y reconciliación.
Equipo completo	Revisar criterios de aceptación, priorizar defectos, aceptar riesgos y aprobar el cierre de cada ciclo.

12. Riesgos de prueba

Riesgo	Mitigación prevista
Cuota o latencia de NVIDIA	Dobles de prueba, muestras pequeñas, checkpoints, deduplicación previa y ventanas controladas.
Fuentes externas variables	Fixtures versionadas, pruebas por familia de conectores, aislamiento por fuente y monitoreo de fallos.
Datos geográficos incompletos	Diferenciar dirección confirmada/aproximada y cubrir eventos sin coordenadas; Maps no bloquea el listado.
Dependencia de servicios gratuitos	Separar pruebas locales y de integración; registrar límites de GitHub, Supabase, Vercel y APIs externas.

Riesgo	Mitigación prevista
Pruebas dependientes de fecha	Usar reloj controlado y datos relativos para evitar fallos al vencer un evento de fixture.
Cambios de esquema	Aplicar migraciones en entorno de prueba y ejecutar regresión antes de sincronizar datos.
Cobertura regional desigual	Preparar datos de varias regiones y verificar filtros aun cuando el volumen real sea bajo.

13. Trazabilidad y mantenimiento

Los casos se vinculan con la **matriz MoSCoW de Fase 2** y con el **backlog de historias de usuario** incluidos entre las evidencias del proyecto. La matriz de casos debe actualizarse cuando cambie un requisito, una fuente, el esquema de Supabase, un contrato de API o una vista del frontend. Las nuevas funciones se incorporan primero como subplan y después como casos concretos, sin declarar resultados antes de ejecutarlos.